STAT 8305: Pattern Recognition
Take-Home Project: Next-Word Prediction

Goal. The purpose of this project is to study a simplified next-word prediction problem. Using the previous three words as input, you will develop and compare linear and nonlinear prediction models, with emphasis on numerical representation, model selection, prediction, and interpretation.

Dataset

Two files are provided:

- `STAT8305_next_word_training.csv`, containing the previous three words and the observed next word;
- `STAT8305_next_word_testing_20.csv`, containing 20 new observations for prediction.

Each training observation has the form

$$(X_{i1}, X_{i2}, X_{i3}) \longrightarrow Y_i,$$

where X_{i1}, X_{i2}, X_{i3} are the previous three words and Y_i is the next word.

For example,

$$(\text{we}, \text{can}, \text{build}) \longrightarrow \text{models}.$$

Note: Do not use pretrained language models, transformers, RNNs, LSTMs, GRUs, or external text data.

Submission Guidelines

Submit a report answering Questions 1–5, with the written report limited to **no more than 2 pages**, together with separate prediction files for Questions 3 and 5 and your complete reproducible analysis code. R examples are provided below, but other programming languages or software may also be used.

The two-page limit applies only to the written report. Prediction files and analysis code do not count toward the two-page limit. The report should be concise and focus on the main modeling choices, model-selection results, prediction results, and interpretation.

Project Questions

Question 1. Numerical representation

Convert the three text predictors into numerical vectors using one-hot encoding. Explain briefly how the representation works and report the dimension of the resulting training design matrix.

You may use:

```
train <- read.csv("STAT8305_next_word_training.csv")
test  <- read.csv("STAT8305_next_word_testing_20.csv")

lev1 <- sort(unique(train$word1))
lev2 <- sort(unique(train$word2))
lev3 <- sort(unique(train$word3))

train$word1 <- factor(train$word1, levels = lev1)
train$word2 <- factor(train$word2, levels = lev2)
train$word3 <- factor(train$word3, levels = lev3)

test$word1 <- factor(test$word1, levels = lev1)
test$word2 <- factor(test$word2, levels = lev2)
test$word3 <- factor(test$word3, levels = lev3)

X_train <- model.matrix(
    ~ word1 + word2 + word3 - 1,
    data = train
)

X_test <- model.matrix(
    ~ word1 + word2 + word3 - 1,
    data = test
)
```

Report: Briefly explain the meaning of the numerical representation, report the dimension of `X_train`, and show the first 10 rows of the original training data together with the corresponding first 10 rows of `X_train`.

Question 2. Linear-model prediction

Construct a linear-model approach for predicting the next word. One possible strategy is to fit a separate 0–1 linear regression for each possible next word and predict the word having

the largest fitted value. Consider at least two reasonable linear specifications, such as using only the most recent word, using all three previous words, or including interaction terms.

Example R functions are

`lm()`, `predict()`, `model.matrix()`.

Report: Briefly describe the candidate linear models you considered and explain how the fitted values are converted into a predicted next word.

Question 3. Linear-model selection and testing prediction

Use part of the training data as a validation set to determine which linear model from Question 2 should be selected. State your model-selection criterion, report the validation performance of the candidate models, justify your final choice, and then use the selected model to predict the next word for all 20 testing observations.

Report: Report the validation results for the candidate linear models and identify the selected model with a brief justification. Submit the predicted next words for all 20 testing observations in a separate file.

Question 4. Nonlinear prediction model

Construct at least one nonlinear prediction model using the numerical predictors from Question 1. You may consider methods such as a support vector machine, decision tree, random forest, boosting method, neural network, or another reasonable nonlinear prediction method.

Choose a method that you believe is appropriate for this problem and consider reasonable tuning parameters or alternative model specifications. You are encouraged to explore a deep learning model if interested; the deep learning analysis may be implemented in R, Python, or another appropriate software environment.

Examples of useful R packages include

`e1071`, `rpart`, `randomForest`, `xgboost`, `keras3`.

Report: State the nonlinear method or methods you considered, briefly explain why they are appropriate for this prediction problem, and describe the main tuning parameters or model specifications considered.

Question 5. Nonlinear-model selection and testing prediction

Use validation performance to select the final nonlinear model from Question 4. Report the validation results, justify your final model choice, and use the selected model to predict the

next word for all 20 testing observations. Compare its predictions and predictive performance with those from the selected linear model in Question 3.

If you also explored a deep learning model, briefly compare its performance with the linear and nonlinear models and discuss whether the additional model complexity leads to meaningful improvement.

Report: Explain and justify the final nonlinear model you selected and briefly compare it with the linear model. If a deep learning model was also considered, summarize its performance briefly. Submit the predicted next words for all 20 testing observations from your selected nonlinear model in a separate file.

Final Remark: Submit exactly four files to iCollege: the two-page report, the Question 3 linear-model predictions, the Question 5 nonlinear-model predictions, and your complete reproducible analysis code.